

XÂY DỰNG HỆ THỐNG SELF-EVOLVING AI AGENTS TRONG XỬ LÝ SỰ CỐ MẠNG VIỄN THÔNG

Nguyễn Thanh Hải

optimus1072005@gmail.com

Lĩnh vực Data Science & AI Engineering - Chương trình Viettel Digital Talent 2026

Nguyễn Duy Hưng¹

hungnd1235@gmail.com

Nguyễn Duy Anh¹

nguyenduyanh821998@gmail.com

Bùi Quang Hưng¹

contact.hung.bq@gmail.com

¹Phòng Giải pháp di động - Trung tâm Nghiên cứu Công nghệ & Chuyển đổi số - Tổng công ty Mạng lưới Viettel

1. GIỚI THIỆU CHUNG

Các mạng viễn thông hiện đại, đặc biệt trong bối cảnh 5G, trung tâm dữ liệu và mạng điều khiển bằng phần mềm, có quy mô lớn, nhiều lớp giao thức và phụ thuộc chặt chẽ giữa thiết bị, dịch vụ, cấu hình định tuyến, topology và chỉ số hiệu năng. Một sự cố nhỏ như sai ASN BGP, IP missing, link down, switch BMv2 dừng hoạt động hoặc rule SDN sai có thể gây mất kết nối diện rộng, suy giảm chất lượng dịch vụ và làm tăng chi phí vận hành. Trong thực tế, xử lý sự cố không chỉ là phát hiện bất thường, mà còn cần định vị thiết bị lỗi, xác định nguyên nhân (Root Cause Analysis - RCA) và đưa ra hướng khắc phục có thể kiểm chứng. Các khảo sát về RCA tự động cho thấy bài toán này khó vì triệu chứng quan sát được thường gián tiếp, chịu nhiễu, và bắt nguồn từ nhiều tầng cấu hình khác nhau [1].

Các cách tiếp cận truyền thống thường dựa trên luật thủ công, kinh nghiệm chuyên gia hoặc mô hình học máy tĩnh. Những phương pháp này khó thích ứng khi topology thay đổi, khi xuất hiện dạng lỗi mới hoặc khi dữ liệu vận hành có nhiễu. Sự phát triển của mô hình ngôn ngữ lớn (LLM) và AI Agent mở ra khả năng mới: agent có thể đọc mô tả sự cố, sử dụng công cụ chẩn đoán, suy luận nhiều bước và giải thích kết quả bằng ngôn ngữ tự nhiên. Các nghiên cứu gần đây về LLM cho chẩn đoán mạng nhấn mạnh tiềm năng kết hợp tri thức miền, log vận hành và công cụ kiểm chứng thay vì chỉ dựa vào câu trả lời một lượt của mô hình [2]. Tuy nhiên, agent thông thường vẫn mang tính tĩnh; nếu prompt, bộ nhớ hoặc mô tả công cụ không được cập nhật thì hiệu quả chẩn đoán dễ suy giảm khi môi trường mạng thay đổi. Vì vậy, hệ thống cần học từ trace gọi công cụ, phản hồi chấm điểm và các case đã xử lý để điều chỉnh quy trình suy luận cho những sự cố tiếp theo.

Đề tài này nghiên cứu và xây dựng hệ thống **Self-Evolving AI Agents** cho xử lý sự cố mạng viễn thông trên nền tảng benchmark NIKA [3]. Khác với agent tĩnh, self-evolving agent khai thác chính lịch sử xử lý nhiệm vụ, phản hồi và kết quả đánh giá làm nguồn dữ liệu để cập nhật bộ nhớ, công cụ hoặc chiến lược suy luận [4]. Mục tiêu của đề tài là biến mỗi lần chẩn đoán thành một nguồn kinh nghiệm giúp agent cải thiện ở các lần sau. Cụ thể, hệ thống cần tiếp nhận dữ liệu từ môi trường mạng giả lập, gọi các công cụ kiểm tra trạng thái mạng, suy luận nguyên nhân gốc rễ, gửi kết quả đánh giá và cập nhật tri thức chẩn đoán dài hạn.

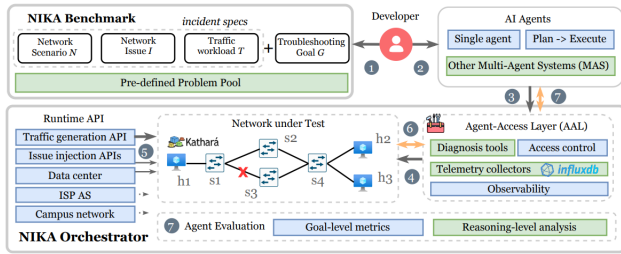
Ý nghĩa thực tiễn của sản phẩm nằm ở việc hỗ trợ kỹ sư vận hành thực hiện nhanh các bước lặp lại như kiểm tra kết nối, đọc cấu hình, đối chiếu bảng định tuyến, phân tích log và tổng hợp bằng chứng. Hệ thống không thay thế chuyên gia, mà đóng vai trò trợ lý chẩn đoán giúp khoanh vùng nguyên nhân, chuẩn hóa báo cáo sự cố và lưu lại kinh nghiệm đã kiểm chứng để tái sử dụng cho các ca tương tự. Đóng góp chính của sinh viên gồm:

- Thiết kế và hiện thực workflow agent chẩn đoán trên nền tảng NIKA/Kathará, tích hợp các dịch vụ MCP để agent tương tác với router, host, switch P4/BMv2, telemetry và hệ thống chấm điểm.
- Đề xuất hai mô-đun tự tiến hóa gồm **Procedural Memory** lấy cảm hứng từ Skill-Pro [5] và **Tool Refinement** lấy cảm hứng từ DRAFT [6], giúp agent lưu trữ kỹ năng chẩn đoán và tinh chỉnh tài liệu công cụ dựa trên trace thực tế.
- Xây dựng pipeline benchmark để so sánh baseline và agent tự tiến hóa trên NIKA, đánh giá theo detection, localization, RCA, số bước, số lần gọi công cụ, tool error rate, token và thời gian chạy.

2. NỘI DUNG VÀ PHƯƠNG PHÁP

2.1. Giới thiệu về benchmark NIKA

NIKA được giới thiệu như một network arena để đánh giá AI Agent trong bài toán xử lý sự cố mạng [7]. Benchmark này giải quyết một khoảng trống quan trọng: trước đây việc đánh giá agent chẩn đoán mạng thường thiếu môi trường chuẩn hóa, khó tái lập và tốn nhiều công sức vận hành. Như minh họa trong Hình 1, NIKA cung cấp các kịch bản mạng có thể replay, giao diện agent-network rõ ràng và tập incident được thiết kế để kiểm tra khả năng phát hiện lỗi, định vị thiết bị lỗi và xác định nguyên nhân gốc rễ. Vì vậy, NIKA phù hợp với mục tiêu của đề tài: xây dựng một agent không chỉ trả lời bằng ngôn ngữ tự nhiên, mà còn phải sử dụng công cụ, thu thập bằng chứng và cải thiện qua nhiều lần xử lý sự cố.



Hình 1. Tổng quan benchmark NIKA cho đánh giá AI Agent trong xử lý sự cố mạng.

2.2. Khung tổng quan hệ thống

Hệ thống được xây dựng quanh một agent chẩn đoán có khả năng tương tác với môi trường mạng thông qua các công cụ MCP. Ở mức tổng quan, môi trường mạng cung cấp topology, thiết bị và dữ liệu quan sát; agent sử dụng các công cụ để kiểm tra trạng thái mạng; bộ chấm điểm đánh giá kết quả cuối cùng; và hai mô-đun tự tiến hóa cập nhật tri thức chẩn đoán sau mỗi episode. Hai mô-đun này gồm Procedural Memory để lưu kinh nghiệm dưới dạng quy trình chẩn đoán và Tool Refinement để cải thiện tài liệu công cụ dựa trên trace thực thi.

2.2.1. Công cụ và công nghệ sử dụng

Hệ thống sử dụng NIKA làm benchmark chính để tạo các incident mạng và chấm điểm kết quả chẩn đoán. Katharà [8] được dùng để ảo hóa topology bằng container, giúp khởi động lab, tiêm lỗi và tái lập môi trường một cách nhất quán. MCP/FastMCP [9] đóng vai trò lớp giao tiếp giữa agent và môi trường mạng: các thao tác như kiểm tra kết nối, đọc cấu hình, truy vấn telemetry hoặc nộp kết quả đều được chuẩn hóa thành công cụ. LangGraph/LangChain [10] được dùng để điều phối workflow agent, trong đó các chiến lược chính gồm ReAct [11], Plan & Execute và Reflexion [12].

Ở tầng dữ liệu và mạng, hệ thống dùng InfluxDB để lưu telemetry, FRRouting [13] cho các kịch bản định tuyến và P4/BMv2 [14] cho các kịch bản data-plane lập trình được. Phần triển khai được viết bằng Python 3.12 và quản lý môi trường bằng uv. Các nhóm công cụ MCP chính gồm Katharà Base, FRR, BMv2/P4, Telemetry, Task Management và Tool Evolution; mỗi nhóm tương ứng với một phần quan sát hoặc điều khiển cần thiết trong quá trình agent xử lý sự cố.

2.2.2. Quy trình triển khai

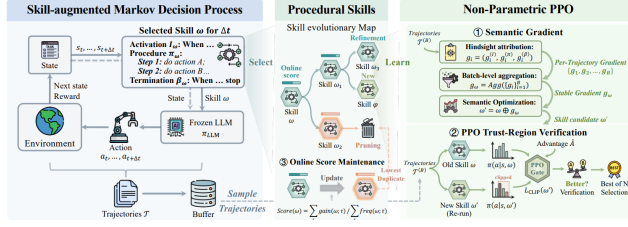
Quy trình triển khai bắt đầu từ việc chọn tập case trong benchmark, chuẩn bị cấu hình lab và xác định loại lỗi cần tiêm. Sau đó hệ thống khởi động môi trường, đưa agent vào vòng chẩn đoán và ghi lại toàn bộ trace gồm suy luận, lời gọi công cụ, quan sát trả về và kết luận cuối cùng. Các trace này vừa phục vụ chấm điểm, vừa là dữ liệu đầu vào cho hai mô-đun tự tiến hóa sau khi episode kết thúc.

Một episode chẩn đoán diễn ra theo trình tự: khởi động lab mạng, tiêm lỗi, chạy agent, thu thập trace gọi công cụ, nộp kết quả gồm `is_anomaly`, `faulty_devices`, `root_cause_name`, sau đó đánh giá và kích hoạt học ngoại tuyến. Với baseline, mỗi episode độc lập. Với evolved agent, bộ nhớ kỹ năng và tài liệu công cụ đã được tinh chỉnh từ các episode trước được nạp lại vào ngữ cảnh để hỗ trợ case tiếp theo.

Luồng xử lý được tách thành hai pha. Trong Diagnosis Phase, agent được quyền dùng đầy đủ công cụ để thu thập bằng chứng, kiểm tra giả thuyết và loại trừ nguyên nhân sai. Trong Submission Phase, agent chỉ còn công cụ nộp kết quả, nhờ đó giảm rủi ro vừa chẩn đoán vừa thay đổi kết luận cuối cùng một cách thiếu kiểm soát. Thiết kế này bám sát tinh thần ReAct, nơi suy luận và hành động công cụ được xen kẽ để giải quyết tác vụ nhiều bước, nhưng bổ sung lớp đánh giá sau episode để phục vụ học liên tục.

2.3. Mô-đun Procedural Memory

Procedural Memory được thiết kế để chuyển kinh nghiệm xử lý sự cố từ dạng lịch sử thụ động sang dạng quy trình có thể tái sử dụng [5]. Trong các incident lặp lại hoặc có cấu trúc tương tự, agent không nên đọc lại toàn bộ trajectory cũ rồi suy luận lại từ đầu. Thay vào đó, hệ thống rút gọn kinh nghiệm thành các kỹ năng chẩn đoán có điều kiện kích hoạt rõ ràng, quy trình thực thi cụ thể và điều kiện dừng. Hình 2 minh họa vòng đời này: trace sau episode được dùng để tạo hoặc cập nhật kỹ năng; ở episode tiếp theo, agent truy xuất kỹ năng phù hợp để định hướng chuỗi hành động công cụ.



Hình 2. Quy trình học kỹ năng tái sử dụng cho mô-đun Procedural Memory.

Mỗi kỹ năng được biểu diễn thành ba thành phần. *Activation condition* mô tả tình huống nên gọi kỹ năng, chẳng hạn mất kết nối host, bất thường định tuyến BGP/OSPF hoặc lỗi liên quan đến switch P4/BMv2. *Execution procedure* mô tả chuỗi bước chẩn đoán cần ưu tiên, ví dụ kiểm tra reachability, đọc cấu hình interface, đối chiếu gateway, kiểm tra bảng định tuyến và xác minh trạng thái neighbor. *Termination condition* mô tả khi nào trả quyền điều khiển lại cho agent, ví dụ khi đã đủ bằng chứng về thiết bị lỗi, khi giả thuyết ban đầu bị loại trừ hoặc khi cần chuyển sang một hướng kiểm tra khác. Nhờ cấu trúc này, memory không chỉ là một đoạn ghi chú, mà trở thành một đơn vị thủ tục có thể được nạp vào quá trình ra quyết định.

Ở đầu mỗi incident, hệ thống tạo mô tả ngữ cảnh từ triệu chứng, topology, giao thức, dịch vụ và các quan sát ban đầu. Bộ truy xuất memory lấy một nhóm kỹ năng ứng viên theo độ phù hợp với ngữ cảnh; bộ chọn memory sau đó ưu tiên kỹ năng có lịch sử sử dụng tốt hơn thay vì đưa toàn bộ kinh nghiệm vào prompt. Trong quá trình chẩn đoán, kỹ năng chỉ đóng vai trò định hướng: agent vẫn phải gọi công cụ, kiểm tra bằng chứng và có thể dừng kỹ năng nếu quan sát thực tế không còn phù hợp với điều kiện kích hoạt ban đầu.

Sau mỗi episode, hệ thống cập nhật Procedural Memory bằng phản hồi từ bộ chấm điểm. Trajectory thành công được dùng để sinh kỹ năng mới hoặc tinh chỉnh quy trình cũ; trajectory thất bại được dùng để sửa điều kiện kích hoạt, thêm điều kiện dừng hoặc loại bỏ bước kiểm tra gây nhiễu. Cơ chế semantic gradient giúp biến kinh nghiệm thành đề xuất sửa kỹ năng, còn PPO gate phi tham số [15] đóng vai trò kiểm soát chất lượng: kỹ năng ứng viên chỉ được nhận nếu có dấu hiệu cải thiện hành vi mà không làm lệch quá mạnh khỏi chiến lược đã có. Ngoài ra, bộ nhớ duy trì điểm chất lượng theo thời gian để ưu tiên kỹ năng ổn định và loại dần các kỹ năng trùng lặp, quá hẹp hoặc ít hữu ích.

2.4. Mô-đun Tool Refinement

Tool Refinement xử lý một nguồn lỗi khác của agent: tài liệu công cụ có thể đúng với kỹ sư vận hành nhưng chưa đủ rõ với LLM khi ra quyết định gọi công cụ [6]. Trong bài toán chẩn đoán mạng, một công cụ thường

phụ thuộc vào loại node, tên container, giao thức đang chạy, trạng thái lab và định dạng đầu ra. Nếu tài liệu thiếu tiền điều kiện, thiếu ví dụ hoặc không giải thích failure mode, agent có thể gọi đúng tên công cụ nhưng sai thiết bị, sai tham số hoặc diễn giải sai kết quả. Vì vậy, mô-đun này chỉ tinh chỉnh tài liệu công cụ; mã nguồn công cụ và quyền thực thi không thay đổi.

Bảng 1. Thuật toán học của mô-đun Tool Refinement

Input	Tập tài liệu công cụ $\mathcal{D}$, số vòng I , ngưỡng tương đồng ϕ , ngưỡng dừng τ .
Output	Tập tài liệu đã tinh chỉnh $\tilde{\mathcal{D}}$.
<pre> 1 $\tilde{\mathcal{D}} \leftarrow \emptyset$. 2 for mỗi tài liệu $t \in \mathcal{D}$ do 3 for $i = 1 \dots I$ do 4 Explorer sinh tình huống khám phá e_i. 5 Nếu e_i quá giống lịch sử theo ϕ, sinh lại truy vấn khác. 6 Thực thi công cụ và ghi nhận kết quả r_i. 7 Analyzer rút lỗi tài liệu và gợi ý sửa đổi s_i. 8 Rewriter cập nhật t_i và hướng khám phá tiếp theo. 9 Nếu độ thay đổi $\Delta(t_{i-1}, t_i) > \tau$, dừng vòng học. 10 end for 11 $\tilde{\mathcal{D}} \leftarrow \tilde{\mathcal{D}} \cup \{t_i\}$. 12 end for 13 return $\tilde{\mathcal{D}}$. </pre>	

Quy trình học của Tool Refinement gồm ba vai trò liên tiếp. Explorer tạo các tình huống sử dụng công cụ từ tài liệu hiện tại, lịch sử khám phá và hướng kiểm tra tiếp theo. Để tránh lặp lại cùng một kiểu truy vấn, Explorer áp dụng ràng buộc tương đồng: nếu tình huống mới quá giống các tình huống trước, hệ thống yêu cầu sinh lại tình huống khác nhằm bao phủ nhiều cách dùng công cụ hơn. Mỗi lần thử ghi lại tham số đã gọi, đầu ra, lỗi phát sinh và ngữ cảnh chẩn đoán để làm dữ liệu học.

Analyzer đọc kết quả thử nghiệm để phát hiện khoảng cách giữa tài liệu và hành vi thực tế của agent, từ đó sinh gợi ý sửa đổi có mục tiêu. Rewriter kết hợp gợi ý, trace thực thi và lịch sử chỉnh sửa để tạo phiên bản tài liệu mới, đồng thời đề xuất hướng khám phá cho vòng tiếp theo. Quá trình dừng khi tài liệu đã hội tụ hoặc khi các vòng thử tiếp theo không còn tạo thêm thay đổi đáng kể. Cách thiết kế này phù hợp với hệ thống mạng vì nó cải thiện khả năng sử dụng công cụ của agent nhưng vẫn giữ ranh giới an toàn: chỉ phần mô tả công cụ tiến hóa theo trải nghiệm, còn công cụ gốc vẫn được giữ nguyên.

3. THỰC NGHIỆM

3.1. Thiết lập thực nghiệm

Toàn bộ thực nghiệm được chạy trên máy CPU 12 cores, 16 GB RAM. Các mô hình ngôn ngữ lớn không được triển khai cục bộ mà được gọi qua Netmind API, gồm GPT-OSS-120B và Qwen3.5-122B-A10B-FP8. Cấu hình chính dùng để báo cáo kết quả là ReAct + GPT-OSS-120B; các workflow Plan&Execute và Reflexion được dùng thêm để so sánh ảnh hưởng của cách tổ chức suy luận. Các thí nghiệm gồm hai chế độ: **Baseline Agent** không nạp kinh nghiệm từ các episode trước, còn **Evolved Agent** bổ sung Procedural Memory và Tool Refinement. Procedural Memory dùng `top_k=5`, `memory 1500 tokens`, `skill_max_age=4`, `skill_pool=32`, LCB selector với `selector_minlcb=-0.05`, heuristic controller với `PP0_epsilon=0.05`, `nominee_k=3` và 3 evolution samples. Tool Refinement dùng `num_docs=6`, `num_scoped_docs=4`, `doc_chars=500`, `planned_check=4`, `next_checks=2` và ngưỡng hội tụ 0.75.

3.2. Kết quả thực nghiệm

Tập benchmark con gồm 125 case, trong đó 100 case được lấy từ bản full và 25 case là no-fault. Các case bao phủ nhiều nhóm lỗi như link failures, end-host failures, misconfigurations, resource contention, network node errors và network under attack. Nhóm metric cơ bản gồm detection, localization, RCA, số token, số lần gọi công cụ, số lần gọi công cụ lỗi và runtime. Với phân tiến hóa, hệ thống ghi nhận thêm các tín hiệu vận hành của memory/tool evolution như kỹ năng được truy xuất, ứng viên kỹ năng được sinh, tài liệu công cụ được cập nhật và trạng thái hội tụ của tài liệu.

Kết quả thực nghiệm được tổng hợp từ log chạy agent, trace gọi công cụ, kết quả nộp cuối cùng và bộ chấm điểm của benchmark. Các bảng và phân tích ở phần tiếp theo trình bày ý nghĩa của kết quả theo hai góc nhìn: hiệu quả chẩn đoán cơ bản và hiệu quả của cơ chế tự tiến hóa.

4. ĐÁNH GIÁ HIỆU QUẢ

4.1. Tiêu chí đánh giá

Basic. Nhóm tiêu chí cơ bản được dùng để đo chất lượng xử lý sự cố của agent ở cả hai mức: độ chính xác chẩn đoán và chi phí vận hành. Detection đo khả năng xác định một case có lỗi hay không; localization đánh giá việc định vị đúng thiết bị hoặc thành phần lỗi; RCA đánh giá khả năng xác định đúng nguyên nhân gốc rễ. Incident success rate được xem là tiêu chí nghiêm ngặt hơn vì yêu cầu agent đúng đồng thời các thành phần chính của một kết luận xử lý sự cố. Bên cạnh đó, số bước suy luận, số lần gọi công cụ, lỗi gọi công cụ, token

đầu vào/đầu ra và runtime được ghi nhận để đánh giá chi phí khi agent đi đến kết luận.

Evolve. Nhóm tiêu chí tiến hóa tập trung vào việc đánh giá liệu Procedural Memory và Tool Refinement có giúp agent cải thiện qua các episode hay không. Với Procedural Memory, quá trình đánh giá xem kỹ năng nào được truy xuất, kỹ năng đó có phù hợp với ngữ cảnh sự cố hay không và việc tái sử dụng kỹ năng có giúp agent thu hẹp không gian kiểm tra hay không. Với Tool Refinement, quá trình đánh giá tập trung vào chất lượng tài liệu công cụ sau khi cập nhật, mức độ giảm lỗi khi gọi công cụ và khả năng hội tụ của tài liệu sau nhiều vòng sửa đổi. Hai nhóm tiêu chí này bổ sung cho nhau: Basic cho biết kết quả cuối cùng của agent, còn Evolve cho biết cơ chế học kinh nghiệm có thật sự tạo ra thay đổi hữu ích trong quá trình vận hành hay không.

4.2. Phân tích chuyên sâu

Kết quả chung trong Bảng 3 cho thấy lợi ích rõ nhất của Evolved Agent nằm ở các chỉ số cần suy luận nhiều bước, đặc biệt là RCA và incident success rate. Điều này phù hợp với vai trò của Procedural Memory: thay vì bắt đầu mỗi incident như một bài toán hoàn toàn mới, agent có thể truy xuất lại các quy trình đã từng hiệu quả trong những case liên quan. Nhờ vậy, agent có xu hướng đi theo lộ trình chẩn đoán có cấu trúc hơn, bắt đầu từ xác nhận triệu chứng, khoanh vùng thiết bị, kiểm tra cấu hình liên quan rồi mới kết luận nguyên nhân gốc rễ. Mức cải thiện ở detection và localization không lớn bằng RCA, nhưng vẫn cho thấy việc tổ chức lại kinh nghiệm giúp agent ổn định hơn trong các bước đầu của quy trình chẩn đoán.

Ở tầng sử dụng công cụ, Tool Refinement giúp giảm các lần gọi công cụ kém giá trị. Trong trace thực nghiệm, baseline đôi khi gọi đúng loại công cụ nhưng sai thiết bị, dùng tham số chưa phù hợp hoặc đọc đầu ra mà không liên hệ được với giả thuyết RCA. Khi tài liệu công cụ được cập nhật bằng preconditions, usage notes, failure modes và ví dụ diễn giải đầu ra, agent có thêm ngữ cảnh để lựa chọn công cụ chính xác hơn. Đây là lý do số bước và số lần gọi công cụ có xu hướng giảm, trong khi lỗi gọi công cụ gần như được loại bỏ ở cấu hình evolved.

Tuy nhiên, cải thiện này không hoàn toàn miễn phí. Việc đưa kỹ năng và tài liệu công cụ đã tinh chỉnh vào ngữ cảnh làm tăng token và kéo dài runtime. Đây là trade-off quan trọng của self-evolving agent: hệ thống đổi thêm chi phí ngữ cảnh để nhận lại quy trình chẩn đoán tốt hơn và ít thao tác sai hơn. Với các case liên quan gần nhau, lợi ích của memory thể hiện sớm vì agent có thể tái sử dụng quy trình đã học. Với các case xa hơn về topology, giao thức hoặc loại lỗi, lợi ích giảm

Bảng 2. Tóm tắt phạm vi benchmark trong thực nghiệm

Nhóm sự cố	Ví dụ	Số lượng
Link Failures	<code>link_down</code> , <code>link_flap</code> , packet corruption	12
End-Host Failures	thiếu IP, sai gateway, sai DNS, IP conflict	16
Misconfigurations	sai ASN BGP, thiếu route, sai OSPF area, flow rule loop, P4 table misconfig	27
Resource Contention	quá tải sender/receiver, incast, load balancer overload	13
Network Node Errors	FRR down, BMv2 switch down, SDN controller crash, MPLS label limit	20
Network Under Attack	BGP hijacking, DHCP spoofing, ARP poisoning, ACL block, web DoS	12
no-fault	mạng bình thường, không tiêm lỗi	25

Bảng 3. Kết quả chung của ReAct trên Baseline và Evolved với GPT-OSS-120B và Qwen3.5-122B-A10B-FP8

Metric	GPT-OSS Baseline	GPT-OSS Evolved	Qwen Baseline	Qwen Evolved
Detection	—	—	—	—
Localization Acc.	—	—	—	—
Localization F1	—	—	—	—
RCA Acc.	—	—	—	—
RCA F1	—	—	—	—
Incident Success	—	—	—	—
Avg. Steps	—	—	—	—
Avg. Tool Calls	—	—	—	—
Tool Error Rate	—	—	—	—
Input Tokens	—	—	—	—
Output Tokens	—	—	—	—
Total Tokens	—	—	—	—
Runtime	—	—	—	—

Bảng 4. Ablation study trên Evolved Agent khi thay đổi workflow

Metric	ReAct	Plan&Execute	Reflexion
Detection	—	—	—
Localization	—	—	—
RCA	—	—	—
Input Token	—	—	—
Output Token	—	—	—
Runtime	—	—	—

dần do kỹ năng cũ ít phù hợp hơn và agent cần quay lại quá trình kiểm chứng bằng công cụ.

Khi so sánh giữa workflow trong Bảng 4, ReAct phù hợp với benchmark hiện tại vì vòng lặp quan sát–hành động ngắn, dễ điều chỉnh giả thuyết sau mỗi kết quả công cụ. Plan&Execute có ưu điểm về cấu trúc kế hoạch nhưng dễ kém linh hoạt khi quan sát thực tế khác với kế hoạch ban đầu. Reflexion bổ sung bước tự phản tư nhưng có thể làm tăng chi phí suy luận nếu phản tư không dẫn đến hành động kiểm chứng mới. So sánh giữa GPT-OSS-120B và Qwen3.5-122B-A10B-FP8 chủ yếu nhằm kiểm tra xem lợi ích của Procedural Memory và Tool Refinement có phụ thuộc hoàn toàn vào một mô hình nền hay không. Kết quả quan sát cho thấy hướng cải thiện vẫn xuất hiện khi thay đổi backbone, dù mức độ có thể khác nhau.

4.3. Hạn chế và rủi ro

Hệ thống vẫn có một số hạn chế. Thứ nhất, hiệu quả tự tiến hóa phụ thuộc vào chất lượng episode trước đó; nếu feedback hoặc ground truth không đáng tin cậy, bộ nhớ có thể ghi nhận quy trình sai và làm nhiều các case sau. Thứ hai, kỹ năng học được từ một nhóm topology có thể quá khớp với tên thiết bị, giao thức hoặc cách tổ chức lab cũ, làm giảm khả năng chuyển sang topology mới. Thứ ba, Tool Refinement cải thiện cách mô tả và sử dụng công cụ, nhưng không thể bù cho việc môi trường thiếu telemetry, log hoặc tín hiệu quan sát cần thiết. Cuối cùng, chi phí token tăng cho thấy cần có cơ chế retrieval và chọn lọc ngữ cảnh chặt chẽ hơn khi triển khai ở quy mô lớn.

Các rủi ro này cho thấy self-evolving agent cần được vận hành theo nguyên tắc có kiểm soát. Trong môi trường mạng quan trọng, hệ thống nên ghi rõ bằng chứng cho từng kết luận, cho phép kỹ sư phê duyệt kỹ năng mới trước khi lưu lâu dài và có cơ chế rollback bộ nhớ khi phát hiện tri thức sai. Đây cũng là lý do đề tài ưu tiên tiến hóa bộ nhớ và tài liệu công cụ thay vì tự động sinh thao tác cấu hình mới trên thiết bị mạng.

5. KẾT LUẬN

Đề tài đã xây dựng được một prototype Self-Evolving AI Agents cho xử lý sự cố mạng viễn thông trên nền tảng NIKA. Hệ thống tích hợp môi trường mạng giả

Placeholder figure: curve Detection, Localization và RCA theo từng batch 25 case.
Cấu hình: ReAct Evolved trên GPT-OSS-120B và Qwen3.5-122B-A10B-FP8.

Hình 3. Diễn biến metric theo từng batch 25 case của ReAct Evolved trên hai mô hình nền.

Placeholder figure: độ ổn định qua 3 lần chạy.
Cấu hình: ReAct trên Baseline và Evolved với GPT-OSS-120B.

Hình 4. So sánh độ ổn định qua ba lần chạy của ReAct Baseline và ReAct Evolved trên GPT-OSS-120B.

lập, các MCP server chẩn đoán, workflow agent và hai mô-đun tự tiến hóa gồm Procedural Memory và Tool Refinement. Kết quả thực nghiệm cho thấy hướng tiếp cận này giúp agent xử lý sự cố có cấu trúc hơn, cải thiện khả năng xác định nguyên nhân gốc rễ và giảm các thao tác công cụ kém hiệu quả mà không cần huấn luyện lại mô hình nền. Tính mới của đề tài nằm ở việc xem agent như một hệ thống có thể học sau mỗi episode thay vì một thành phần tĩnh chỉ phụ thuộc vào prompt ban đầu. Procedural Memory giúp lưu lại quy trình chẩn đoán có điều kiện kích hoạt, còn Tool Refinement giúp cải thiện cách agent hiểu và sử dụng công cụ dựa trên trace thực tế. Hai mô-đun này tạo thành một vòng lặp học kinh nghiệm phù hợp với bài toán vận hành mạng, nơi mỗi sự cố đã xử lý có thể trở thành nguồn tri thức cho các sự cố tương tự về sau. Về đóng góp cá nhân, sinh viên đã trực tiếp nghiên cứu các hướng self-evolving agent, thiết kế kiến trúc tích hợp với NIKa, hiện thực hai mô-đun học kinh nghiệm, xây dựng pipeline benchmark và phân tích kết quả. Trong các bước phát triển tiếp theo, hệ thống cần được tối ưu

khả năng chọn lọc memory, giảm chi phí token, tăng khả năng tổng quát hóa sang topology/giao thức mới và bổ sung cơ chế human-in-the-loop để kiểm soát tri thức được lưu lâu dài. Khi các điểm này được hoàn thiện, hệ thống có thể phát triển thành trợ lý chẩn đoán hỗ trợ kỹ sư vận hành mạng trong việc thu thập bằng chứng, khoanh vùng nguyên nhân và chuẩn hóa báo cáo sự cố.

TÀI LIỆU THAM KHẢO

- [1] Yingying Liu et al. A survey on automated network root cause analysis. *arXiv preprint*, 2023. [1](#)
- [2] Shuai Zhou et al. Llm-based network fault diagnosis: A survey. *arXiv preprint*, 2024. [1](#)
- [3] Stefano Martiradonna et al. Nika: Network intelligence for knowledge-driven agents – a benchmark for ai-based network troubleshooting. *arXiv preprint*, 2025. [1](#)
- [4] Huan-ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Yiran Wu, Hongru Wang, Han Xiao, Yuhang Zhou, Shaokun Zhang, Jiayi Zhang, Jinyu Xiang, Yixiong Fang, Qiwen Zhao, Dongrui Liu, Qihan Ren, Cheng Qian, Zhenhailong Wang, Minda Hu,

- Huazheng Wang, Qingyun Wu, Heng Ji, and Mengdi Wang. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence. *arXiv preprint arXiv:2507.21046*, 2025. URL <https://arxiv.org/abs/2507.21046>. 1
- [5] Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. Skill-pro: Learning reusable skills from experience via non-parametric ppo for llm agents. *arXiv preprint arXiv:2602.01869*, 2026. URL <https://arxiv.org/abs/2602.01869>. 1, 2
- [6] Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. From exploration to mastery: Enabling llms to master tools via self-driven interactions. *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2410.08197>. 1, 3
- [7] Zhihao Wang, Alessandro Cornacchia, Alessio Sacco, Franco Galante, Marco Canini, and Dingde Jiang. A network arena for benchmarking ai agents on network troubleshooting. *arXiv preprint arXiv:2512.16381*, 2025. URL <https://arxiv.org/abs/2512.16381>. 2
- [8] Stefano Marrone, Luca Nicoletti, and Claudio Pizzuti. Kathará: a container-based framework for implementing network function virtualization and software-defined networking. In *IEEE International Conference on Computer Communication and Networks*, pages 1–9, 2019. 2
- [9] Anthropic. Model context protocol. <https://modelcontextprotocol.io>, 2024. 2
- [10] Harrison Chase et al. Langgraph: Building stateful, multi-actor applications with llms. *LangChain Documentation*, 2024. 2
- [11] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. 2
- [12] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023. 2
- [13] FRRouting Project. Frouting: A free range routing protocol suite. *GitHub Repository*, 2021. 2
- [14] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. P4: Programming protocol-independent packet processors. In *ACM SIGCOMM Computer Communication Review*, volume 44, pages 87–95, 2014. 2
- [15] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. 3